

Lab Experiments
Set(26-30)
Date:17-08-2026

26.Dataset: Student Mini Data

Create Student_Mini_Data.csv

Code:

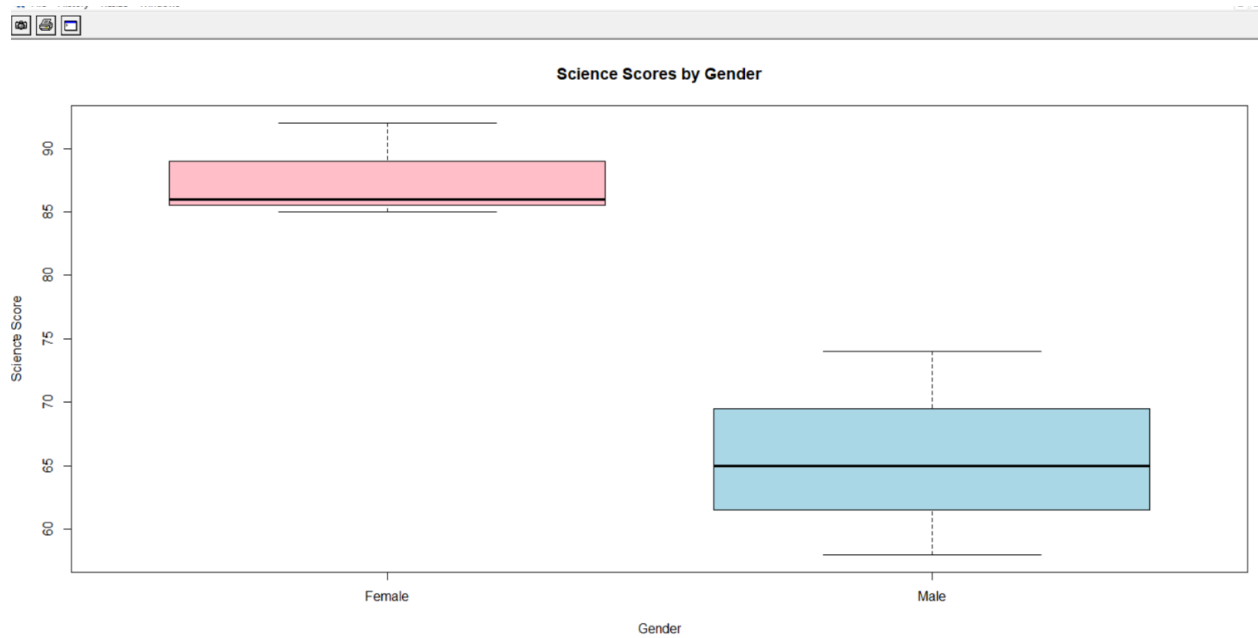
```
student_data <- data.frame(  
  Student_ID = c("S01","S02","S03","S04","S05","S06"),  
  Gender = c("Male","Female","Male","Female","Male","Female"),  
  Study_Hours = c(2.0,3.5,1.5,4.0,2.8,3.0),  
  Attendance = c(78,90,70,95,85,92),  
  Math_Score = c(62,80,55,90,72,82),  
  Science_Score = c(65,85,58,92,74,86),  
  Exam_Date = c("2025-01-10","2025-01-10","2025-02-12",  
    "2025-02-12","2025-03-15","2025-03-15")  
)  
write.csv(student_data, "Student_Mini_Data.csv", row.names = FALSE)
```

1. Import dataset, Histogram and Boxplot

R Program Code:

```
student <- read.csv("Student_Mini_Data.csv")  
print(student)  
hist(student$Math_Score,  
  main = "Histogram of Math Scores",  
  xlab = "Math Score",  
  ylab = "Number of Students",  
  col = "skyblue",  
  border = "black")  
boxplot(Science_Score ~ Gender,  
  data = student,  
  main = "Science Scores by Gender",  
  xlab = "Gender",  
  ylab = "Science Score",  
  col = c("pink", "lightblue"))
```

Output:

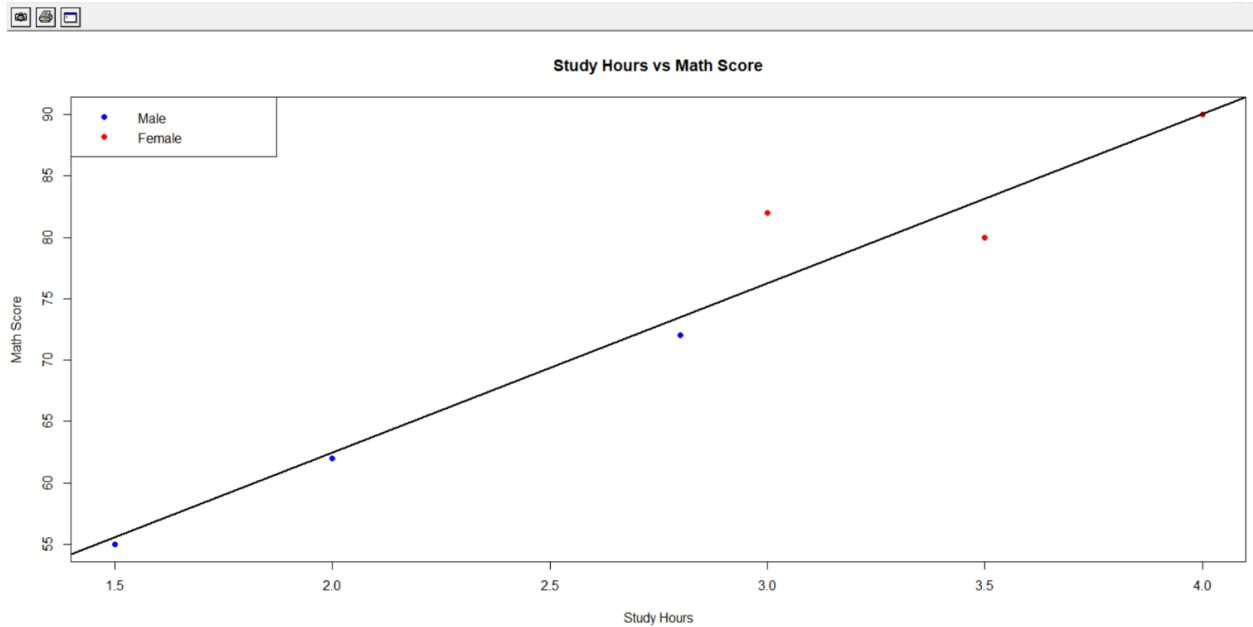


2.Scatter plot: Study Hours vs Math Score

R Program Code:

```
plot(student$Study_Hours,
      student$Math_Score,
      col = ifelse(student$Gender == "Male", "blue", "red"),
      pch = 19,
      main = "Study Hours vs Math Score",
      xlab = "Study Hours",
      ylab = "Math Score")
model <- lm(Math_Score ~ Study_Hours, data = student)
abline(model,
       col = "black",
       lwd = 2)
legend("topleft",
      legend = c("Male", "Female"),
      col = c("blue", "red"),
      pch = 19)
```

Output:



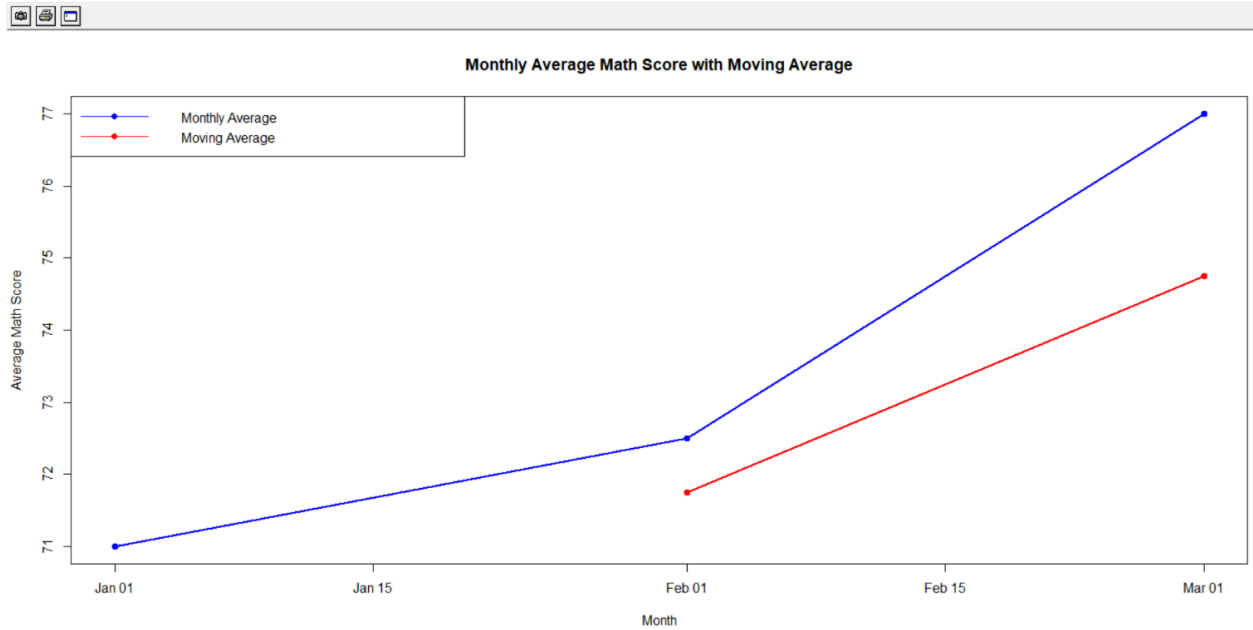
3.Convert date, calculate monthly average and moving average

R Program Code:

```
student <- read.csv("Student_Mini_Data.csv")
student$Exam_Date <- as.Date(student$Exam_Date)
student$Month <- format(student$Exam_Date, "%Y-%m")
monthly_avg <- aggregate(
  Math_Score ~ Month,
  data = student,
  FUN = mean
)
print(monthly_avg)
monthly_avg$Month_Date <- as.Date(
  paste0(monthly_avg$Month, "-01")
)
monthly_avg$Moving_Average <- c(
  NA,
  (monthly_avg$Math_Score[1] +
   monthly_avg$Math_Score[2]) / 2,
  (monthly_avg$Math_Score[2] +
   monthly_avg$Math_Score[3]) / 2
)
```

```
print(monthly_avg)
plot(
  monthly_avg$Month_Date,
  monthly_avg$Math_Score,
  type = "o",
  pch = 19,
  col = "blue",
  lwd = 2,
  main = "Monthly Average Math Score with Moving Average",
  xlab = "Month",
  ylab = "Average Math Score"
)
lines(
  monthly_avg$Month_Date[2:3],
  monthly_avg$Moving_Average[2:3],
  type = "o",
  pch = 19,
  col = "red",
  lwd = 2
)
legend(
  "topleft",
  legend = c("Monthly Average", "Moving Average"),
  col = c("blue", "red"),
  lty = 1,
  pch = 19
)
```

Output:



4.R Shiny code

R Program Code:

```
library(shiny)
library(ggplot2)
library(dplyr)
student <- read.csv("Student_Mini_Data.csv")
student$Exam_Date <- as.Date(student$Exam_Date)
ui <- fluidPage(
  titlePanel("Student Performance Dashboard"),
  sidebarLayout(
    sidebarPanel(
      dateRangeInput(
        "date",
        "Select Exam Date:",
        start = min(student$Exam_Date),
        end = max(student$Exam_Date)
      )
    ),
    mainPanel(
      h3("Average Math Score by Gender"),
      plotOutput("genderPlot"),
      h3("Math Score Trend Over Time"),
      plotOutput("trendPlot")
    )
  )
)
```

```

)
)
server <- function(input, output) {
  filtered_data <- reactive({
    student %>%
      filter(
        Exam_Date >= input$date[1],
        Exam_Date <= input$date[2]
      )
  })
  output$genderPlot <- renderPlot({
    avg_data <- filtered_data() %>%
      group_by(Gender) %>%
      summarise(
        Average_Math = mean(Math_Score)
      )
    ggplot(avg_data,
      aes(x = Gender,
          y = Average_Math,
          fill = Gender)) +
    geom_bar(stat = "identity") +
    labs(
      title = "Average Math Score by Gender",
      x = "Gender",
      y = "Average Math Score"
    ) +
    theme_minimal()
  })
  output$trendPlot <- renderPlot({
    trend_data <- filtered_data() %>%
      group_by(Exam_Date) %>%
      summarise(
        Average_Math = mean(Math_Score)
      )
    ggplot(trend_data,
      aes(x = Exam_Date,
          y = Average_Math)) +
    geom_line(linewidth = 1) +
    geom_point(size = 3) +
    labs(

```

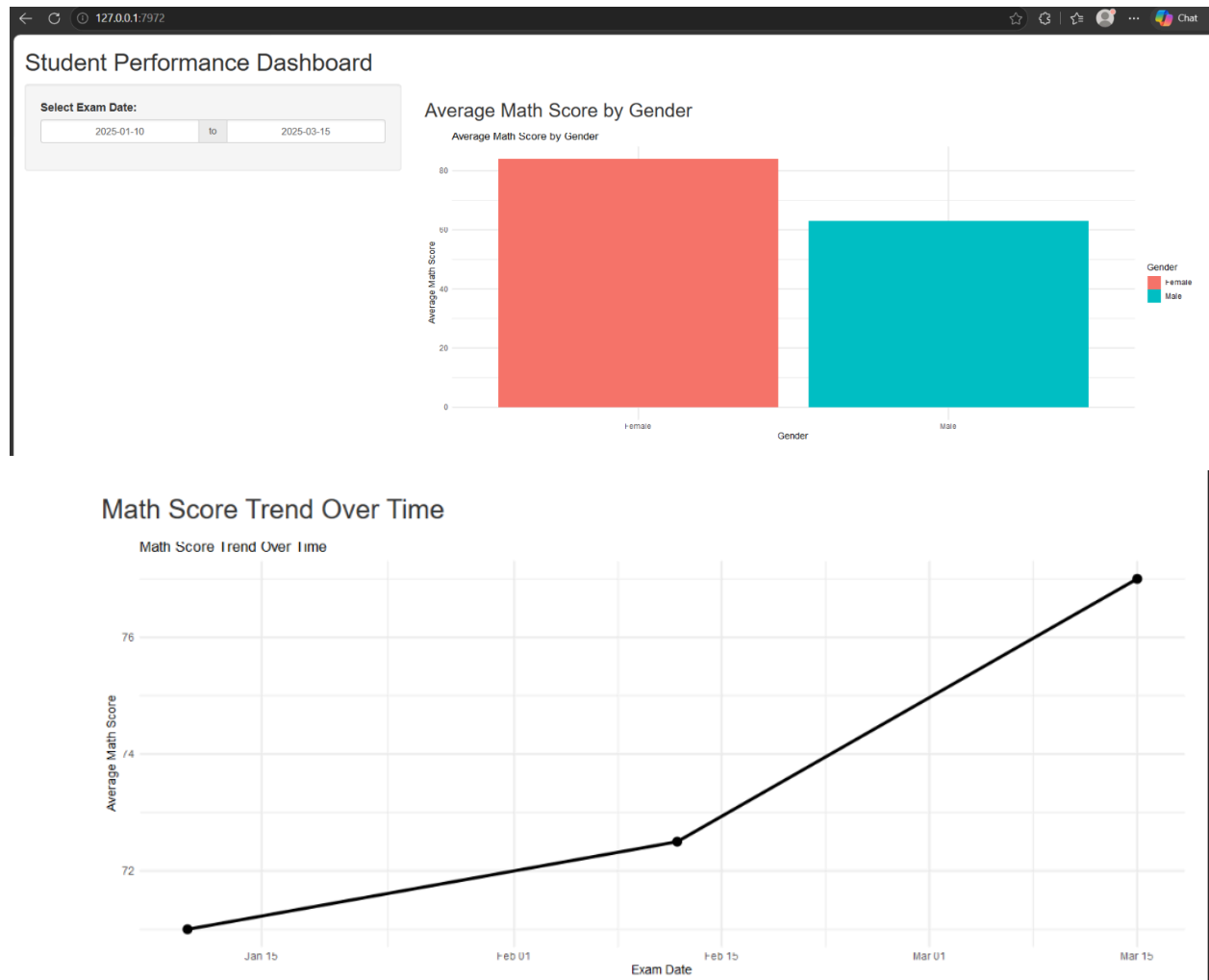
```

    title = "Math Score Trend Over Time",
    x = "Exam Date",
    y = "Average Math Score"
  ) +
  theme_minimal()

})
}
shinyApp(ui = ui, server = server)

```

Output:



SET 27. Patient Health Risk Analysis

1. Scatterplot Matrix + Q-Q Plot + ECDF

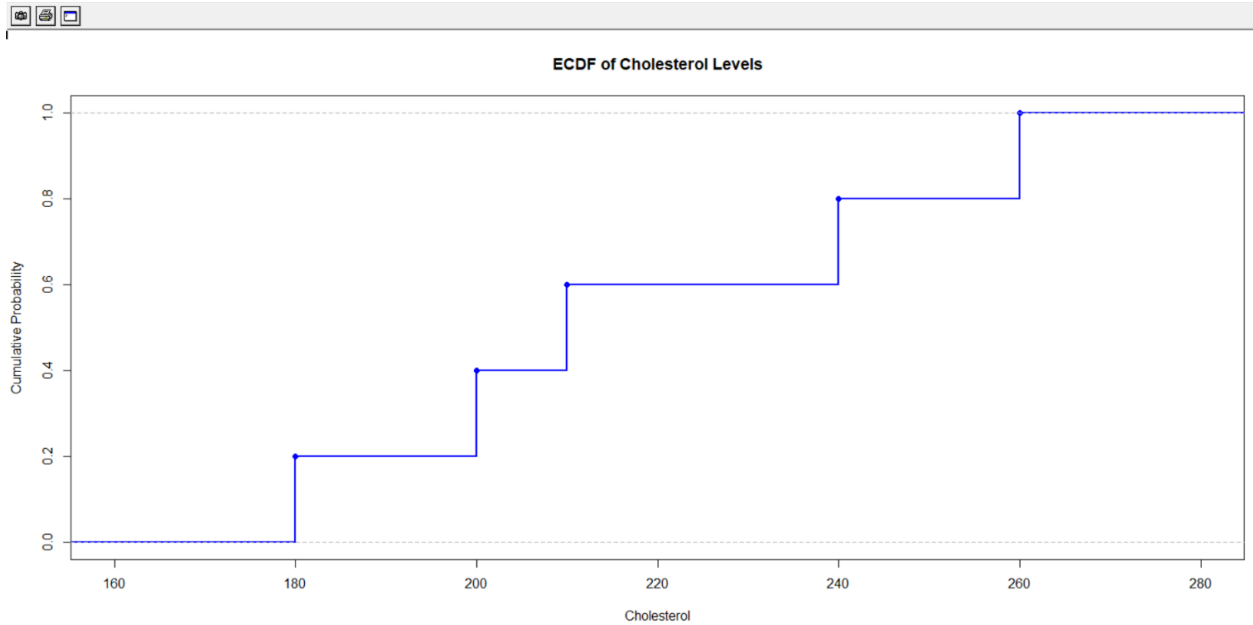
R Program Code:

```

library(GGally)
library(ggplot2)
library(dplyr)
patient <- data.frame(
  Patient_ID = c("P1", "P2", "P3", "P4", "P5"),
  Age = c(25, 40, 55, 35, 60),
  BMI = c(22, 28, 30, 26, 32),
  BP = c(120, 135, 145, 130, 150),
  Cholesterol = c(180, 210, 240, 200, 260)
)
print(patient)
ggpairs(
  patient[, c("Age", "BMI", "BP", "Cholesterol")],
  title = "Patient Health Indicators - Scatterplot Matrix"
)
qqnorm(
  patient$Cholesterol,
  main = "Q-Q Plot of Cholesterol Levels",
  pch = 19
)
qqline(
  patient$Cholesterol,
  col = "red",
  lwd = 2
)
plot(
  ecdf(patient$Cholesterol),
  main = "ECDF of Cholesterol Levels",
  xlab = "Cholesterol",
  ylab = "Cumulative Probability",
  verticals = TRUE,
  do.points = TRUE,
  col = "blue",
  lwd = 2
)

```

Output:

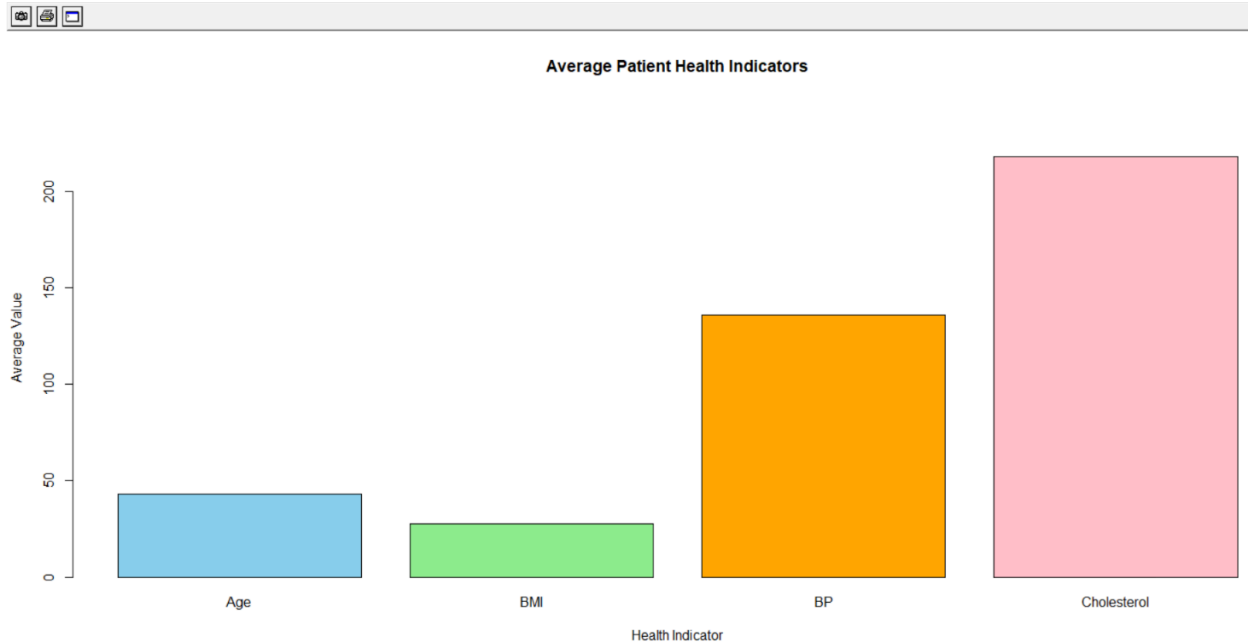


2. Average Health Indicators Bar Chart

R Program Code:

```
averages <- c(  
  Age = mean(patient$Age),  
  BMI = mean(patient$BMI),  
  BP = mean(patient$BP),  
  Cholesterol = mean(patient$Cholesterol)  
)  
print(averages)  
barplot(  
  averages,  
  col = c("skyblue", "lightgreen", "orange", "pink"),  
  main = "Average Patient Health Indicators",  
  xlab = "Health Indicator",  
  ylab = "Average Value",  
  ylim = c(0, max(averages) + 30)  
)
```

Output:



3.In R, instead of Tableau.

R Program Code:

```
library(shiny)
library(ggplot2)
library(dplyr)
patient <- data.frame(
  Patient_ID = c("P1","P2","P3","P4","P5"),
  Age = c(25,40,55,35,60),
  BMI = c(22,28,30,26,32),
  BP = c(120,135,145,130,150),
  Cholesterol = c(180,210,240,200,260)
)
patient$Risk_Score <-
  (patient$BMI >= 30) +
  (patient$BP >= 140) +
  (patient$Cholesterol >= 240)
ui <- fluidPage(
  titlePanel("Patient Health Risk Analysis Dashboard"),
  sidebarLayout(
    sidebarPanel(
      sliderInput(
        "age",
        "Select Age Range:",
```

```

    min = min(patient$Age),
    max = max(patient$Age),
    value = c(min(patient$Age), max(patient$Age)),
    step = 1
  ),
  h4("Risk Score"),
  p("0 = Lower risk indicators"),
  p("1 = Moderate"),
  p("2 = High"),
  p("3 = Highest")
),
mainPanel(
  h3("Patient Risk Heatmap"),
  plotOutput("heatmap"),
  h3("Patient Health Details"),
  tableOutput("details")
)
)
)
server <- function(input, output) {
  filtered_data <- reactive({
    patient %>%
      filter(
        Age >= input$age[1],
        Age <= input$age[2]
      )
  })
  output$heatmap <- renderPlot({
    data <- filtered_data()
    ggplot(
      data,
      aes(
        x = Patient_ID,
        y = "Risk",
        fill = Risk_Score
      )
    ) +
    geom_tile(
      color = "white",
      height = 1,

```

```

      width = 0.9
    ) +
    geom_text(
      aes(label = Risk_Score),
      color = "black",
      size = 6
    ) +
    scale_fill_gradient(
      low = "lightgreen",
      high = "red"
    ) +
    labs(
      title = "Patient Risk Heatmap",
      x = "Patient",
      y = "",
      fill = "Risk Score"
    ) +
    theme_minimal() +
    theme(
      axis.text.y = element_blank(),
      axis.ticks.y = element_blank()
    )
  })
  output$details <- renderTable({
    filtered_data() %>%
    select(
      Patient_ID,
      Age,
      BMI,
      BP,
      Cholesterol,
      Risk_Score
    )
  })
}
shinyApp(ui = ui, server = server)

```

Output:



4. Tableau-style interactive dashboard using R Shiny + Plotly.

R Program Code:

```
library(shiny)
library(plotly)
library(dplyr)
patient <- data.frame(
  Patient_ID = c("P1", "P2", "P3", "P4", "P5"),
  Age = c(25, 40, 55, 35, 60),
  BMI = c(22, 28, 30, 26, 32),
  BP = c(120, 135, 145, 130, 150),
  Cholesterol = c(180, 210, 240, 200, 260)
)
patient$Risk_Score <-
  (patient$BMI >= 30) +
  (patient$BP >= 140) +
  (patient$Cholesterol >= 240)
patient$Risk_Level <- ifelse(
  patient$Risk_Score >= 2,
  "High Risk",
  ifelse(patient$Risk_Score == 1,
    "Moderate Risk",
    "Low Risk")
)
ui <- fluidPage(
```

```

titlePanel("Patient Health Risk Analysis Dashboard"),
sidebarLayout(
  sidebarPanel(
    sliderInput(
      "age",
      "Select Age Range:",
      min = min(patient$Age),
      max = max(patient$Age),
      value = c(min(patient$Age), max(patient$Age)),
      step = 1
    ),
    h4("Risk Criteria"),
    p("BMI >= 30 : 1 point"),
    p("BP >= 140 : 1 point"),
    p("Cholesterol >= 240 : 1 point")
  ),
  mainPanel(
    h3("Patient Risk Heatmap"),
    plotlyOutput(
      "heatmap",
      height = "500px"
    ),
    h3("Patient Health Information"),
    tableOutput("patient_table")
  )
)
)
server <- function(input, output) {
  filtered_data <- reactive({
    patient %>%
      filter(
        Age >= input$age[1],
        Age <= input$age[2]
      )
  })
  output$heatmap <- renderPlotly({
    data <- filtered_data()
    plot_ly(
      data = data,
      x = ~Patient_ID,

```

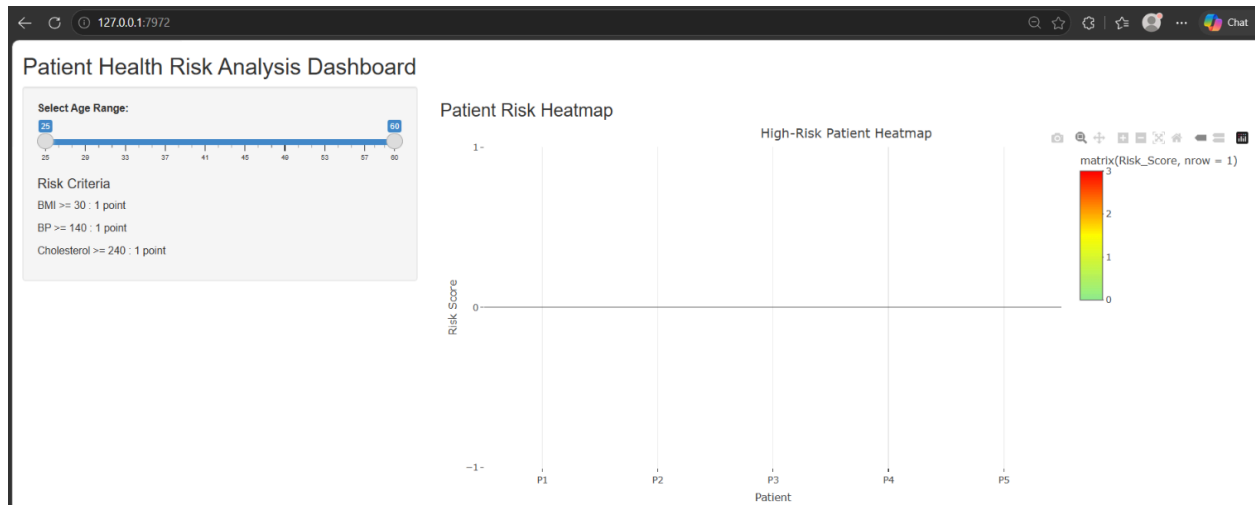
```

y = ~Risk_Score,
type = "heatmap",
z = ~matrix(Risk_Score, nrow = 1),
colorscale = list(
  c(0, "lightgreen"),
  c(0.5, "yellow"),
  c(1, "red")
),
text = ~paste(
  "Patient:", Patient_ID,
  "<br>Age:", Age,
  "<br>BMI:", BMI,
  "<br>BP:", BP,
  "<br>Cholesterol:", Cholesterol,
  "<br>Risk Score:", Risk_Score,
  "<br>Risk Level:", Risk_Level
),
hoverinfo = "text"
) %>%
layout(
  title = "High-Risk Patient Heatmap",
  xaxis = list(title = "Patient"),
  yaxis = list(
    title = "Risk Score",
    dtick = 1
  )
)
})
output$patient_table <- renderTable({
  filtered_data() %>%
  select(
    Patient_ID,
    Age,
    BMI,
    BP,
    Cholesterol,
    Risk_Score,
    Risk_Level
  )
})

```

```
}
shinyApp(ui = ui, server = server)
```

Output:



Patient Health Information

Patient_ID	Age	BMI	BP	Cholesterol	Risk_Score	Risk_Level
P1	25.00	22.00	120.00	180.00	0	Low Risk
P2	40.00	28.00	135.00	210.00	0	Low Risk
P3	55.00	30.00	145.00	240.00	3	High Risk
P4	35.00	26.00	130.00	200.00	0	Low Risk
P5	60.00	32.00	150.00	260.00	3	High Risk

SET 28 – Vehicle Performance Analysis

First: Create the dataset in R

```
vehicle <- data.frame(
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
```



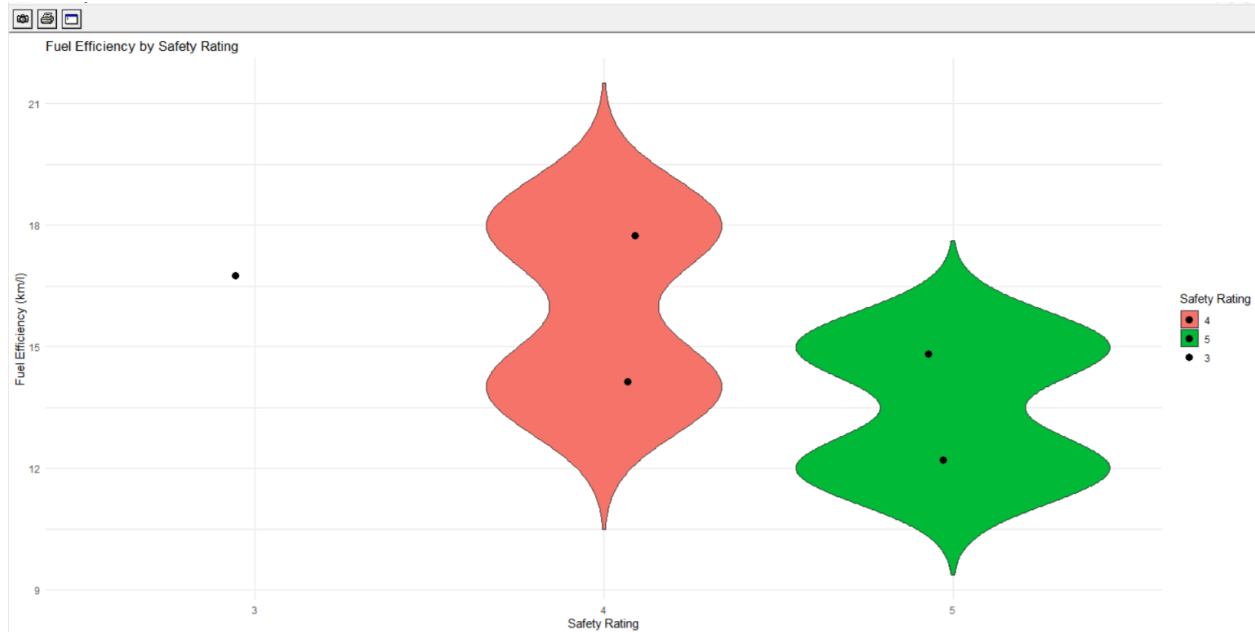
```
Safety_Rating = c(4, 5, 5, 4, 3)
)
print(vehicle)
```

1 . Violin Plot

R Program Code:

```
library(ggplot2)
ggplot(
  vehicle,
  aes(
    x = factor(Safety_Rating),
    y = Fuel_Efficiency,
    fill = factor(Safety_Rating)
  )
) +
  geom_violin(
    trim = FALSE
  ) +
  geom_jitter(
    width = 0.1,
    size = 3
  ) +
  labs(
    title = "Fuel Efficiency by Safety Rating",
    x = "Safety Rating",
    y = "Fuel Efficiency (km/l)",
    fill = "Safety Rating"
  ) +
  theme_minimal()
```

Output:



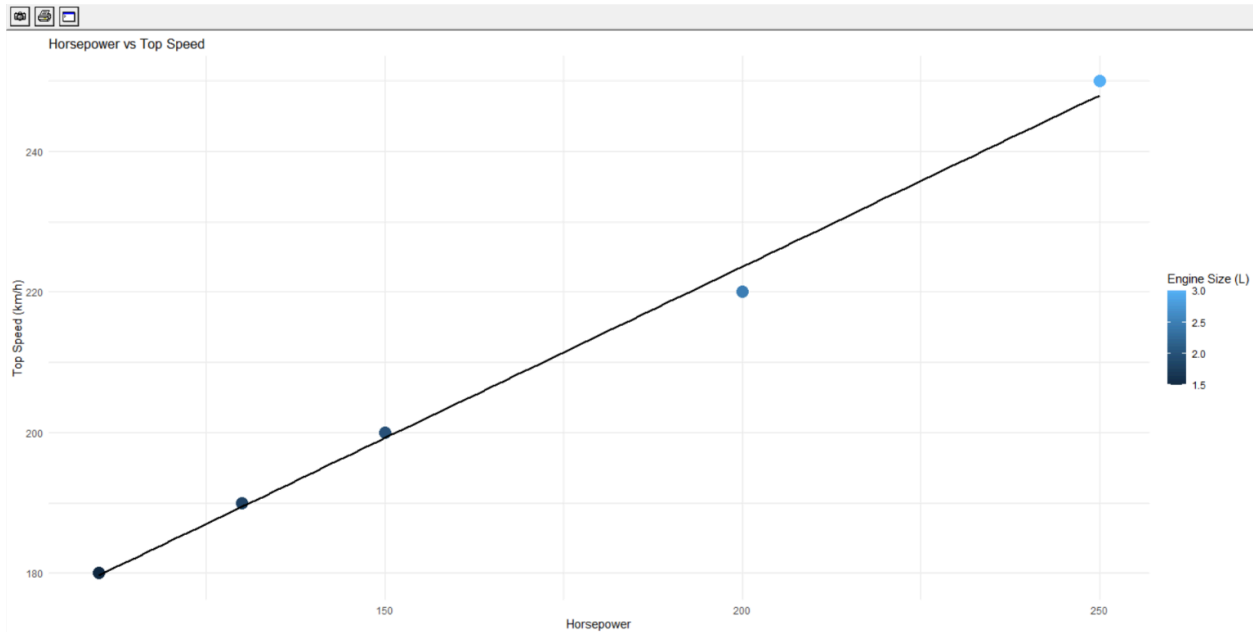
Question 2 — Horsepower vs Top Speed

R Program Code:

```
ggplot(
  vehicle,
  aes(
    x = Horsepower,
    y = Top_Speed,
    color = Engine_Size
  )
) +
  geom_point(
    size = 5
  ) +
  geom_smooth(
    method = "lm",
    se = FALSE,
    color = "black"
  ) +
  labs(
    title = "Horsepower vs Top Speed",
    x = "Horsepower",
    y = "Top Speed (km/h)",
    color = "Engine Size (L)"
  )
```

```
) +  
theme_minimal()
```

Output:



3 . Correlation Heatmap

R Program Code:

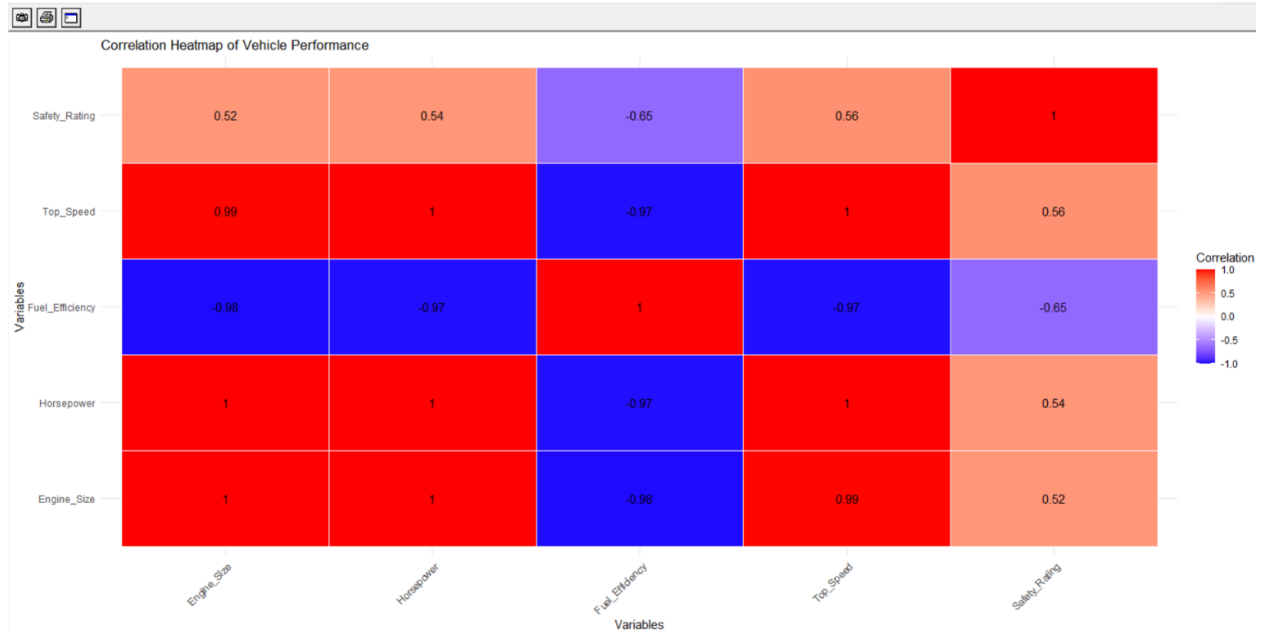
```
vehicle <- data.frame(  
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),  
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),  
  Horsepower = c(110, 150, 250, 200, 130),  
  Fuel_Efficiency = c(18, 15, 12, 14, 17),  
  Top_Speed = c(180, 200, 250, 220, 190),  
  Safety_Rating = c(4, 5, 5, 4, 3)  
)  
num_data <- vehicle[, c(  
  "Engine_Size",  
  "Horsepower",  
  "Fuel_Efficiency",  
  "Top_Speed",  
  "Safety_Rating"  
)]  
cor_matrix <- cor(num_data)  
print(cor_matrix)  
library(ggplot2)
```

```

cor_data <- as.data.frame(as.table(cor_matrix))
colnames(cor_data) <- c("Variable1", "Variable2", "Correlation")
ggplot(
  cor_data,
  aes(
    x = Variable1,
    y = Variable2,
    fill = Correlation
  )
) +
  geom_tile(color = "white") +
  geom_text(
    aes(label = round(Correlation, 2)),
    size = 4
  ) +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0,
    limits = c(-1, 1)
  ) +
  labs(
    title = "Correlation Heatmap of Vehicle Performance",
    x = "Variables",
    y = "Variables",
    fill = "Correlation"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )
)

```

Output:



Question 4 — Interactive Dashboard in R Shiny

R Program Code:

```
library(shiny)
library(plotly)
library(dplyr)
vehicle <- data.frame(
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
  Safety_Rating = c(4, 5, 5, 4, 3)
)
ui <- fluidPage(
  titlePanel(
    "Vehicle Performance Analysis Dashboard"
  ),
  sidebarLayout(
    sidebarPanel(
      h4("Filters"),
      sliderInput(
        "engine",
        "Engine Size (L):",
        min = min(vehicle$Engine_Size),
        max = max(vehicle$Engine_Size),
```

```

    value = c(
      min(vehicle$Engine_Size),
      max(vehicle$Engine_Size)
    ),
    step = 0.1
  ),
  checkboxGroupInput(
    "safety",
    "Safety Rating:",
    choices = sort(unique(vehicle$Safety_Rating)),
    selected = sort(unique(vehicle$Safety_Rating))
  )
),
mainPanel(
  h3("Fuel Efficiency"),
  plotlyOutput(
    "barplot",
    height = "400px"
  ),
  h3("Horsepower vs Top Speed"),
  plotlyOutput(
    "scatterplot",
    height = "400px"
  ),
  h3("Vehicle Information"),
  tableOutput("vehicle_table")
)
)
)
server <- function(input, output) {
  filtered_data <- reactive({
    vehicle %>%
      filter(
        Engine_Size >= input$engine[1],
        Engine_Size <= input$engine[2],
        Safety_Rating %in% input$safety
      )
  })
  output$barplot <- renderPlotly({
    data <- filtered_data()
  })
}

```

```

plot_ly(
  data,
  x = ~Vehicle_ID,
  y = ~Fuel_Efficiency,
  type = "bar",
  text = ~paste(
    "Vehicle:", Vehicle_ID,
    "<br>Engine Size:", Engine_Size, "L",
    "<br>Fuel Efficiency:", Fuel_Efficiency, "km/l",
    "<br>Safety Rating:", Safety_Rating
  ),
  hoverinfo = "text"
) %>%
  layout(
    title = "Fuel Efficiency by Vehicle",
    xaxis = list(
      title = "Vehicle"
    ),
    yaxis = list(
      title = "Fuel Efficiency (km/l)"
    )
  )
)
})
output$scatterplot <- renderPlotly({
  data <- filtered_data()
  plot_ly(
    data,
    x = ~Horsepower,
    y = ~Top_Speed,
    type = "scatter",
    mode = "markers",
    marker = list(size = 12),
    text = ~paste(
      "Vehicle:", Vehicle_ID,
      "<br>Engine Size:", Engine_Size, "L",
      "<br>Horsepower:", Horsepower,
      "<br>Top Speed:", Top_Speed, "km/h",
      "<br>Fuel Efficiency:", Fuel_Efficiency, "km/l",
      "<br>Safety Rating:", Safety_Rating
    ),
  ),

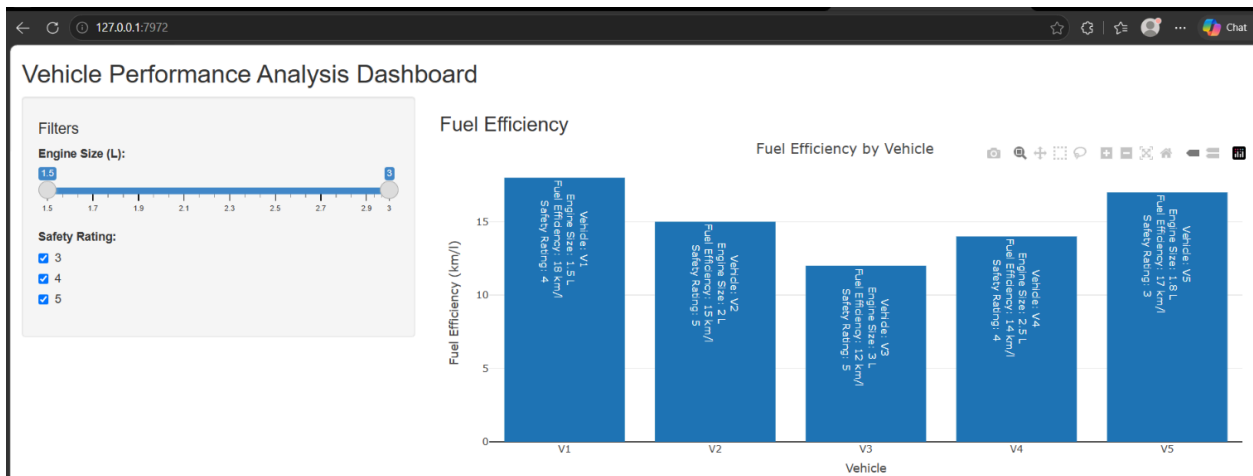
```

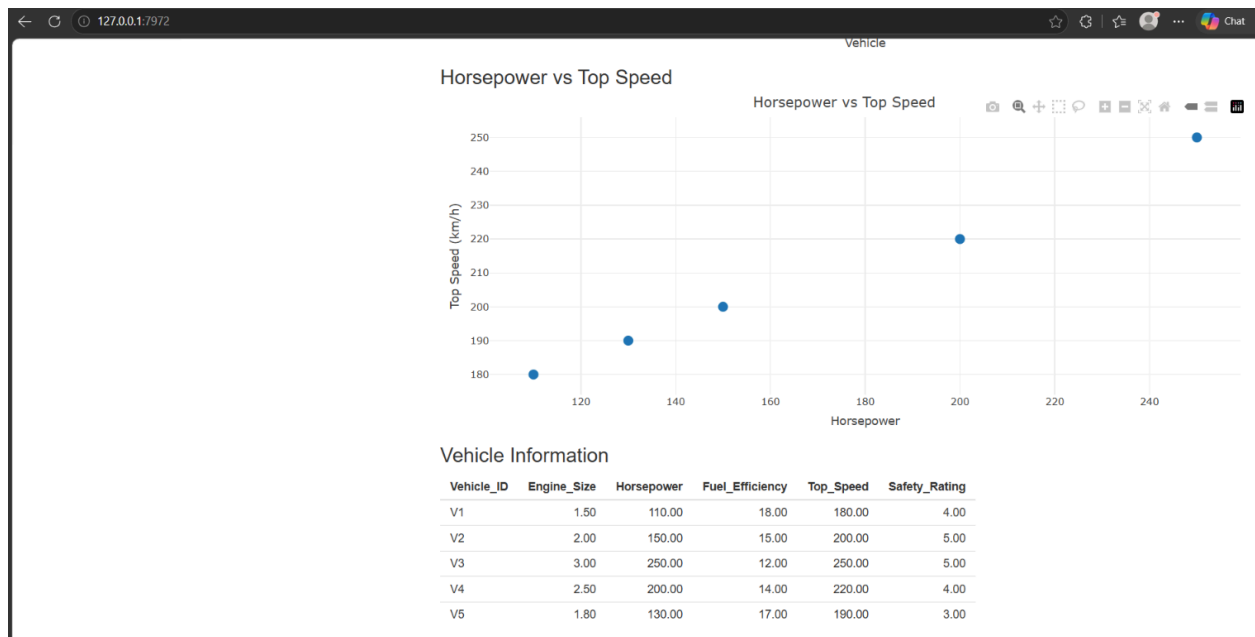
```

    hoverinfo = "text"
  ) %>%
  layout(
    title = "Horsepower vs Top Speed",
    xaxis = list(
      title = "Horsepower"
    ),
    yaxis = list(
      title = "Top Speed (km/h)"
    )
  )
})
output$vehicle_table <- renderTable({
  filtered_data()
})
}
shinyApp(
  ui = ui,
  server = server
)

```

Output:





SET 29 – Student Academic Performance

First: Create the dataset

```
student <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Age = c(19, 21, 20, 22, 23),
  Study_Hours = c(12, 8, 15, 10, 7),
  Attendance = c(90, 70, 95, 85, 60),
  Test_Score = c(85, 70, 92, 80, 65),
  Participation_Score = c(8, 7, 9, 8, 6)
)
print(student)
```

1.Stacked Area Chart

R Program Code:

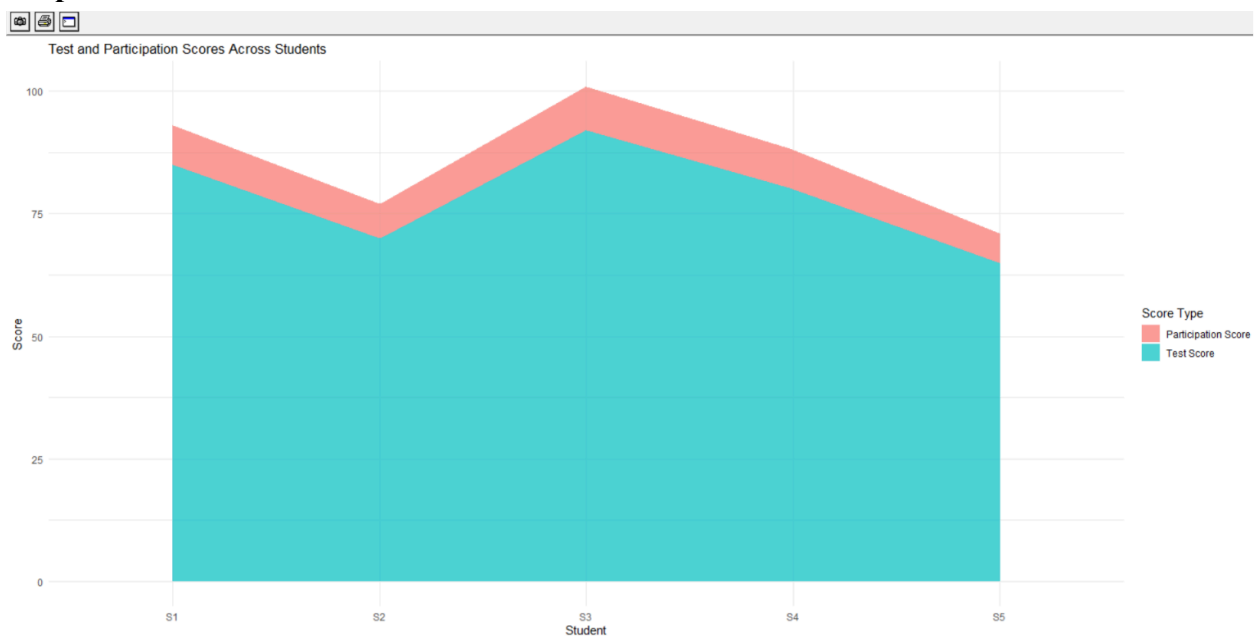
```
library(ggplot2)
area_data <- data.frame(
  Student_ID = rep(student$Student_ID, 2),
  Score_Type = c(
    rep("Test Score", 5),
    rep("Participation Score", 5)
  ),
)
```

```

Score = c(
  student$Test_Score,
  student$Participation_Score
)
)
ggplot(
  area_data,
  aes(
    x = Student_ID,
    y = Score,
    fill = Score_Type,
    group = Score_Type
  )
) +
  geom_area(alpha = 0.7) +
  labs(
    title = "Test and Participation Scores Across Students",
    x = "Student",
    y = "Score",
    fill = "Score Type"
  ) +
  theme_minimal()

```

Output:

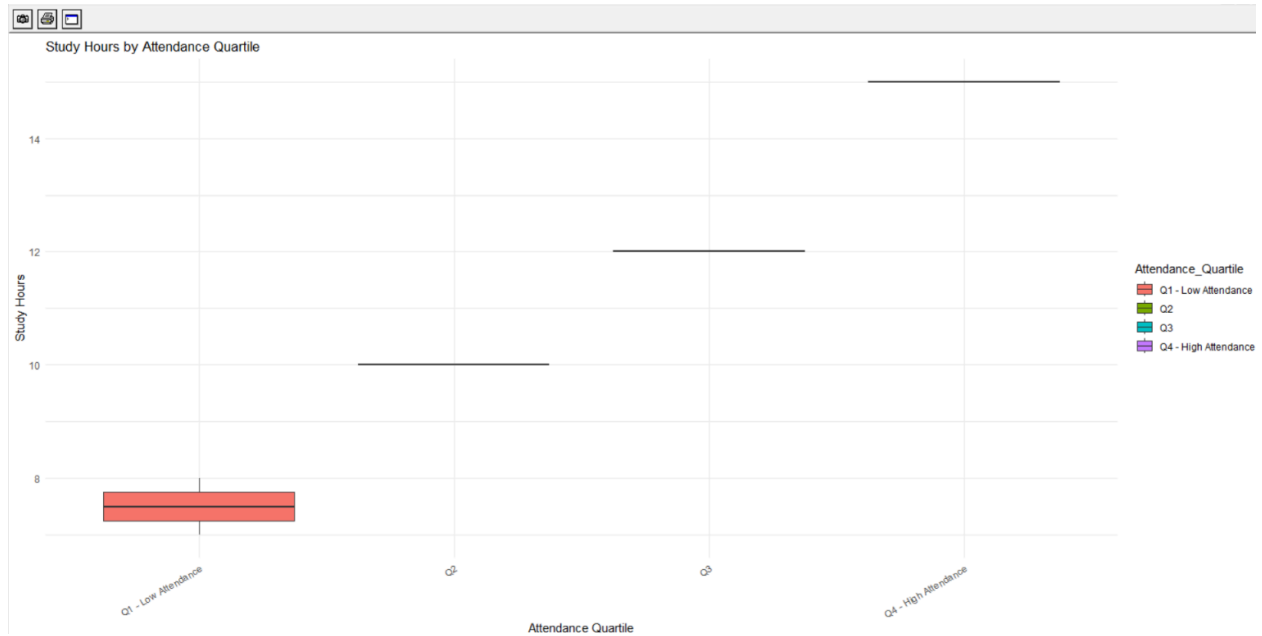


2. Boxplot by Attendance Quartiles

R Program Code:

```
library(ggplot2)
student$Attendance_Quartile <- cut(
  student$Attendance,
  breaks = quantile(
    student$Attendance,
    probs = c(0, 0.25, 0.50, 0.75, 1)
  ),
  include.lowest = TRUE,
  labels = c(
    "Q1 - Low Attendance",
    "Q2",
    "Q3",
    "Q4 - High Attendance"
  )
)
print(student)
ggplot(
  student,
  aes(
    x = Attendance_Quartile,
    y = Study_Hours,
    fill = Attendance_Quartile
  )
) +
  geom_boxplot() +
  labs(
    title = "Study Hours by Attendance Quartile",
    x = "Attendance Quartile",
    y = "Study Hours"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 30,
      hjust = 1
    )
  )
)
```

Output:

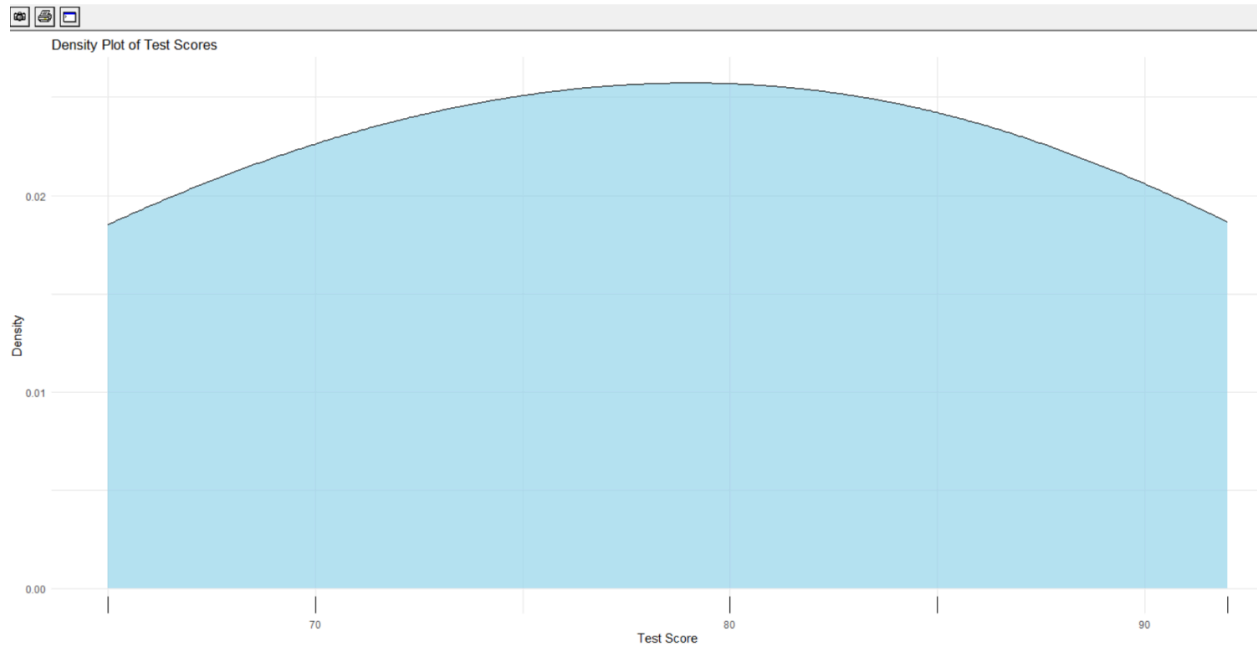


Question 3 — Density Plot

R Program Code:

```
library(ggplot2)
ggplot(
  student,
  aes(
    x = Test_Score
  )
) +
  geom_density(
    fill = "skyblue",
    alpha = 0.6,
    adjust = 1.5
  ) +
  geom_rug(
    sides = "b"
  ) +
  labs(
    title = "Density Plot of Test Scores",
    x = "Test Score",
    y = "Density"
  ) +
  theme_minimal()
```

Output:



4.Interactive Scatterplot

R Program Code:

```
library(shiny)
library(plotly)
student_data <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Age = c(19, 21, 20, 22, 23),
  Study_Hours = c(12, 8, 15, 10, 7),
  Attendance = c(90, 70, 95, 85, 60),
  Test_Score = c(85, 70, 92, 80, 65),
  Participation_Score = c(8, 7, 9, 8, 6)
)
ui <- fluidPage(
  titlePanel("Student Academic Performance Dashboard"),
  sidebarLayout(
    sidebarPanel(
      sliderInput(
        inputId = "age",
        label = "Select Age Range:",
        min = min(student_data$Age),
        max = max(student_data$Age),
```

```

      value = c(
        min(student_data$Age),
        max(student_data$Age)
      ),
      step = 1
    )
  ),
  mainPanel(
    h3("Study Hours vs Test Score"),
    plotlyOutput(
      "scatter_plot",
      height = "500px"
    ),
    br(),
    h3("Student Information"),
    tableOutput("student_table")
  )
)
)
server <- function(input, output, session) {
  filtered_data <- reactive({
    student_data[
      student_data$Age >= input$age[1] &
      student_data$Age <= input$age[2],
    ]
  })
  output$scatter_plot <- renderPlotly({
    data <- filtered_data()
    plot_ly(
      data = data,
      x = ~Study_Hours,
      y = ~Test_Score,
      type = "scatter",
      mode = "markers",
      marker = list(
        size = 14,
        color = ~Participation_Score,
        colorscale = list(
          c(0, "blue"),
          c(0.5, "green"),

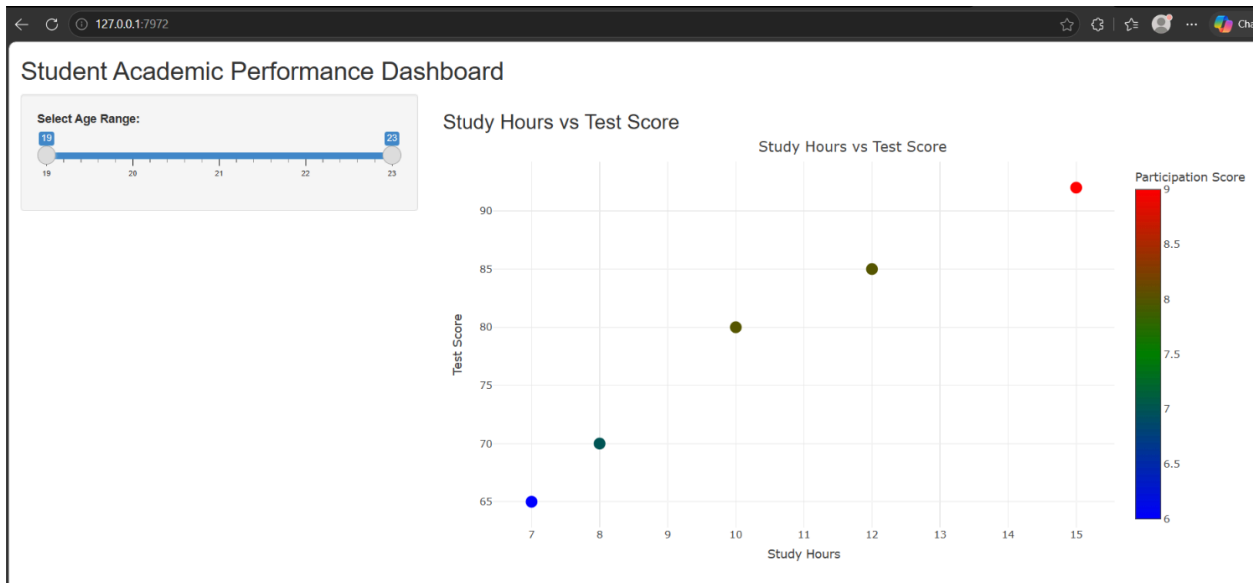
```

```

      c(1, "red")
    ),
    showscale = TRUE,
    colorbar = list(
      title = "Participation Score"
    )
  ),
  text = ~paste(
    "Student:", Student_ID,
    "<br>Age:", Age,
    "<br>Study Hours:", Study_Hours,
    "<br>Test Score:", Test_Score,
    "<br>Participation Score:", Participation_Score
  ),
  hoverinfo = "text"
) %>%
layout(
  title = "Study Hours vs Test Score",
  xaxis = list(
    title = "Study Hours"
  ),
  yaxis = list(
    title = "Test Score"
  )
)
})
output$student_table <- renderTable({
  filtered_data()
})
}
shinyApp(
  ui = ui,
  server = server
)

```

Code:



Student Information

Student_ID	Age	Study_Hours	Attendance	Test_Score	Participation_Score
S1	19.00	12.00	90.00	85.00	8.00
S2	21.00	8.00	70.00	70.00	7.00
S3	20.00	15.00	95.00	92.00	9.00
S4	22.00	10.00	85.00	80.00	8.00
S5	23.00	7.00	60.00	65.00	6.00

SET 30 Dataset: Mobile_App_Usage.csv

Q1. Histogram + Density Plot for Screen_Time

R Program Code:

```
library(ggplot2)
mobile <- data.frame(
  User_ID = c("U01","U02","U03","U04","U05","U06"),
  Gender = c("Male","Female","Male","Female","Male","Female"),
  Age = c(20,22,19,21,23,20),
  Screen_Time = c(4.5,6.0,3.2,7.1,2.8,5.4),
  App_Usage_Count = c(18,25,12,30,10,22),
  Data_Used = c(2.4,3.8,1.6,4.5,1.2,3.1),
  Satisfaction = c(3,5,3,5,2,4),
  Usage_Date = as.Date(c(
```

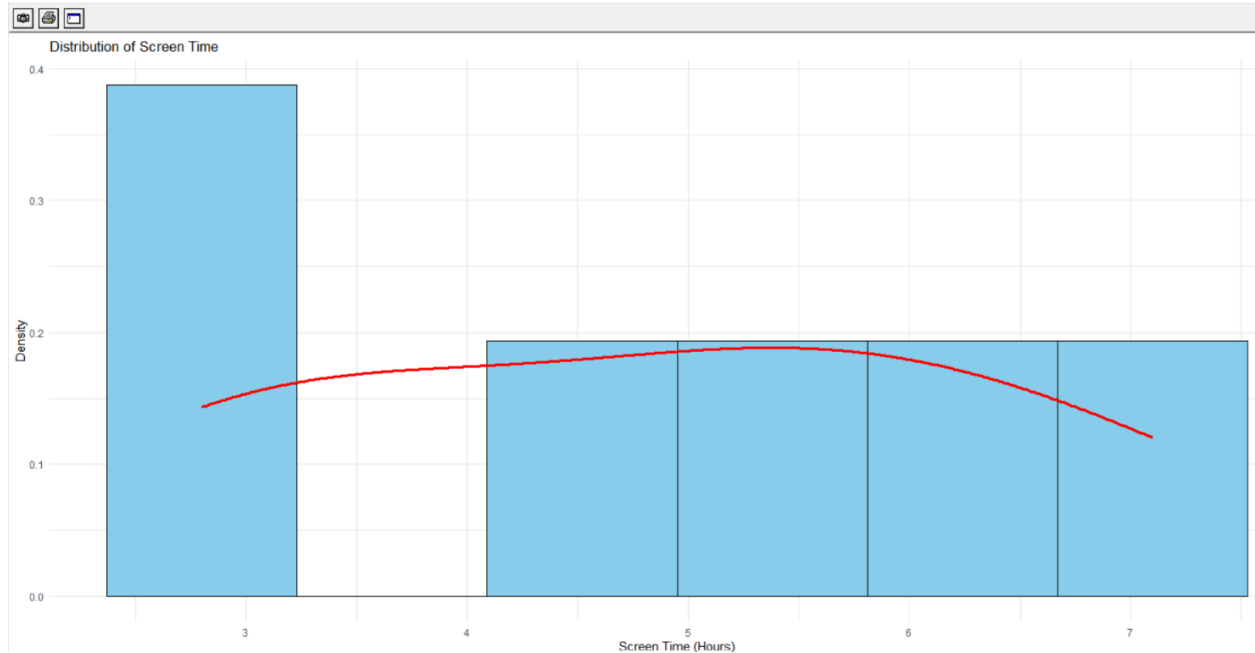


```

    "2025-01-08",
    "2025-01-08",
    "2025-02-11",
    "2025-02-11",
    "2025-03-14",
    "2025-03-14"
  ))
)
ggplot(mobile, aes(x = Screen_Time)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 6,
    fill = "skyblue",
    color = "black"
  ) +
  geom_density(
    color = "red",
    linewidth = 1.2
  ) +
  labs(
    title = "Distribution of Screen Time",
    x = "Screen Time (Hours)",
    y = "Density"
  ) +
  theme_minimal()

```

Output:

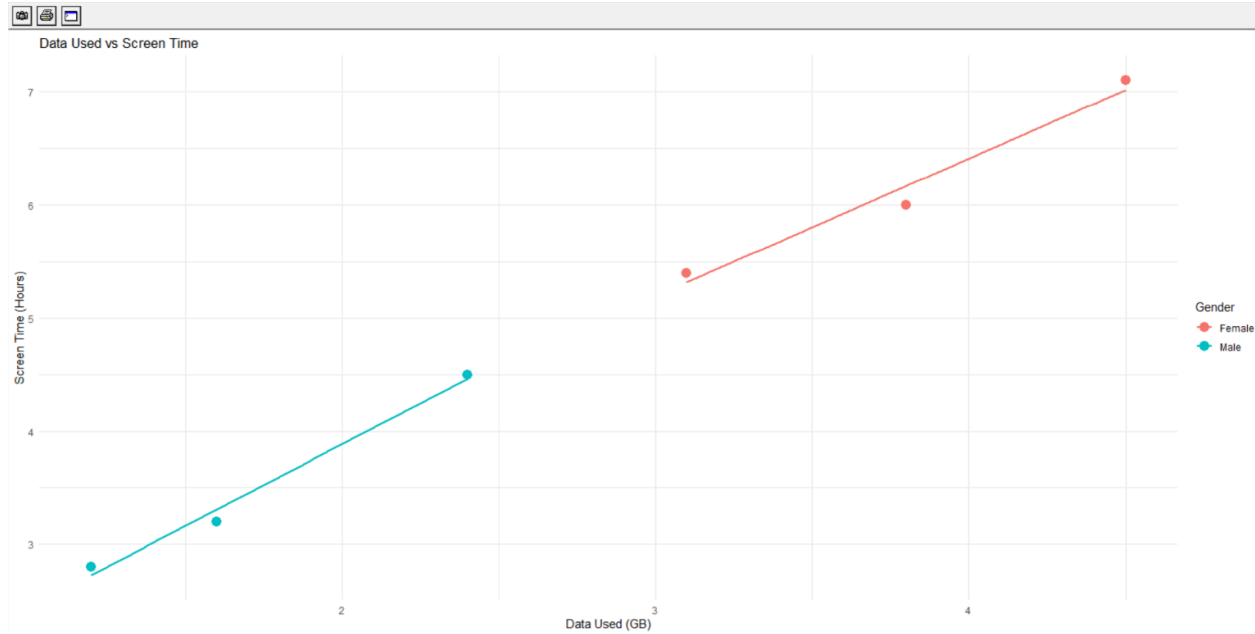


Q2: Data Used vs Screen Time

R Program Code:

```
library(ggplot2)
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),
  Screen_Time = c(4.5, 6.0, 3.2, 7.1, 2.8, 5.4),
  Data_Used = c(2.4, 3.8, 1.6, 4.5, 1.2, 3.1)
)
correlation <- cor(mobile$Data_Used, mobile$Screen_Time)
print(correlation)
ggplot(mobile, aes(x = Data_Used, y = Screen_Time, color = Gender)) +
  geom_point(size = 4) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    title = "Data Used vs Screen Time",
    x = "Data Used (GB)",
    y = "Screen Time (Hours)",
    color = "Gender"
  ) +
  theme_minimal()
```

Output:



Q3: Average Satisfaction Score by Gender + Bar Chart

R Program Code:

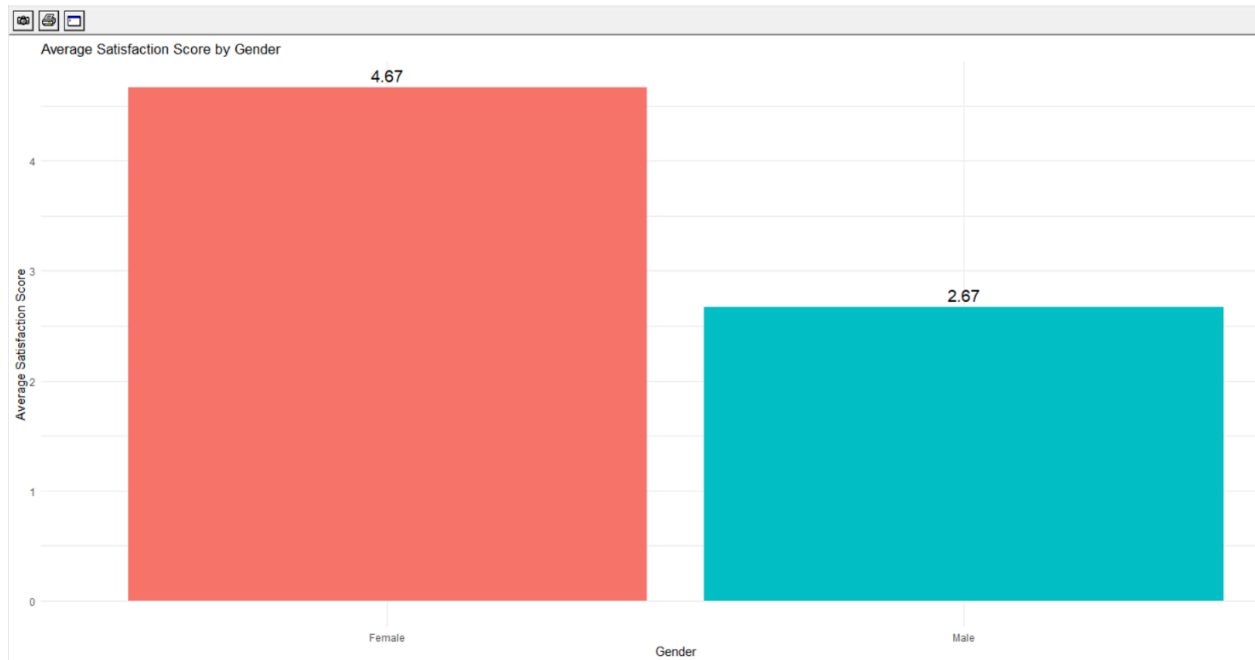
```
library(ggplot2)
library(dplyr)
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),
  Satisfaction = c(3, 5, 3, 5, 2, 4)
)
avg_satisfaction <- mobile %>%
  group_by(Gender) %>%
  summarise(
    Average_Satisfaction = mean(Satisfaction)
  )
print(avg_satisfaction)
ggplot(
  avg_satisfaction,
  aes(
    x = Gender,
    y = Average_Satisfaction,
    fill = Gender
  )
)
```

```

)+
  geom_col() +
  geom_text(
    aes(label = round(Average_Satisfaction, 2)),
    vjust = -0.5,
    size = 5
  ) +
  labs(
    title = "Average Satisfaction Score by Gender",
    x = "Gender",
    y = "Average Satisfaction Score"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none"
  )
)

```

Output:



4.Interactive dashboard using R Shiny + Plotly.

R Program Code:

```

library(shiny)
library(plotly)
mobile <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),

```

```

Gender = c("Male","Female","Male","Female","Male","Female"),
Age = c(20,22,19,21,23,20),
Screen_Time = c(4.5,6.0,3.2,7.1,2.8,5.4),
App_Usage_Count = c(18,25,12,30,10,22),
Data_Used = c(2.4,3.8,1.6,4.5,1.2,3.1),
Satisfaction = c(3,5,3,5,2,4),
Usage_Date = as.Date(c(
  "2025-01-08",
  "2025-01-08",
  "2025-02-11",
  "2025-02-11",
  "2025-03-14",
  "2025-03-14"
))
)
ui <- fluidPage(
  titlePanel("Mobile App Usage Dashboard"),
  sidebarLayout(
    sidebarPanel(
      h4("Filters"),
      checkboxGroupInput(
        "gender",
        "Select Gender:",
        choices = c("Male", "Female"),
        selected = c("Male", "Female")
      ),
      dateRangeInput(
        "date",
        "Select Usage Date:",
        start = min(mobile$Usage_Date),
        end = max(mobile$Usage_Date),
        min = min(mobile$Usage_Date),
        max = max(mobile$Usage_Date)
      )
    ),
    mainPanel(
      h3("Bubble Chart: Screen Time vs Data Used"),
      plotlyOutput(
        "bubble",
        height = "500px"
      )
    )
  )
)

```

```

    ),
    h3("Usage Trend"),
    plotlyOutput(
      "trend",
      height = "400px"
    )
  )
)
)
)
server <- function(input, output, session) {
  filtered_data <- reactive({
    mobile[
      mobile$Gender %in% input$gender &
      mobile$Usage_Date >= input$date[1] &
      mobile$Usage_Date <= input$date[2],
    ]
  })
  output$bubble <- renderPlotly({
    data <- filtered_data()
    plot_ly(
      data = data,
      x = ~Screen_Time,
      y = ~Data_Used,
      type = "scatter",
      mode = "markers",
      color = ~Gender,
      size = ~App_Usage_Count,
      sizes = c(15, 50),
      text = ~paste(
        "User:", User_ID,
        "<br>Gender:", Gender,
        "<br>Screen Time:", Screen_Time, "hours",
        "<br>Data Used:", Data_Used, "GB",
        "<br>App Usage Count:", App_Usage_Count,
        "<br>Satisfaction:", Satisfaction
      ),
      hoverinfo = "text"
    ) %>%
    layout(
      title = "Screen Time vs Data Used",

```

```

      xaxis = list(
        title = "Screen Time (Hours)"
      ),
      yaxis = list(
        title = "Data Used (GB)"
      )
    )
  })
output$trend <- renderPlotly({
  data <- filtered_data()
  plot_ly(
    data = data,
    x = ~Usage_Date,
    y = ~Screen_Time,
    type = "scatter",
    mode = "lines+markers",
    text = ~paste(
      "User:", User_ID,
      "<br>Date:", Usage_Date,
      "<br>Screen Time:", Screen_Time, "hours"
    ),
    hoverinfo = "text"
  ) %>%
  layout(
    title = "Screen Time Usage Trend",
    xaxis = list(
      title = "Usage Date"
    ),
    yaxis = list(
      title = "Screen Time (Hours)"
    )
  )
})
}
shinyApp(
  ui = ui,
  server = server
)

```

Output:

Mobile App Usage Dashboard

Filters

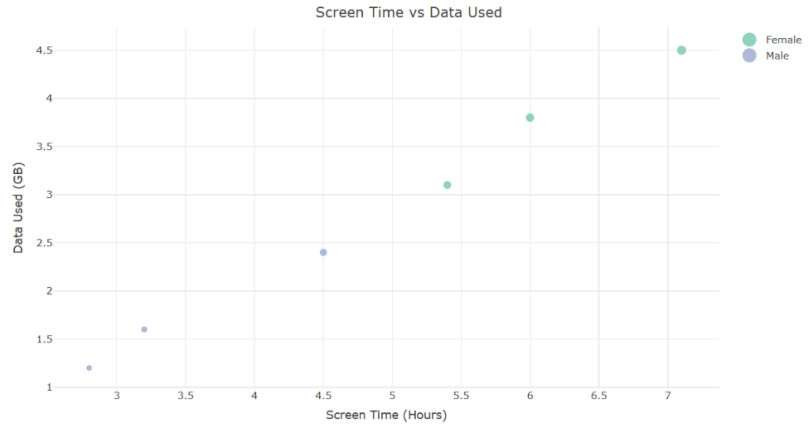
Select Gender:

- ☒ Male
☒ Female

Select Usage Date:

2025-01-08 to 2025-03-14

Bubble Chart: Screen Time vs Data Used



Usage Trend

Screen Time Usage Trend

